TypeScript

Polymorfismus, rozhraní a SOLID

DELTA - Střední škola informatiky a ekonomie, s.r.o.

Ing. Luboš Zápotočný

20.10.2026

CC BY-NC-SA 4.0

Úvod

Navazujeme na základy

Minule jsme si ukázali typový systém, třídy, zapouzdření a dědičnost

Dědičnost ale není cíl – je to prostředek k **polymorfismu**

Dnes nás čeká:

- polymorfismus a přetěžování metod
- abstraktní třídy a rozhraní
- SOLID principy pro návrh tříd

Polymorfismus

Polymorfismus

Polymorfismus znamená „mnoho forem“

Objekt může mít více forem a chovat se různě podle kontextu

V OOP: stejné rozhraní, různá implementace

Příklad polymorfismu

```
class Animal {
  makeSound(): void {
    console.log("Some sound");
  }
}

class Dog extends Animal {
  makeSound(): void {
    console.log("Woof!");
  }
}
```

Příklad polymorfismu

```
class Cat extends Animal {
  makeSound(): void {
    console.log("Meow!");
  }
}

function makeAnimalSound(animal: Animal): void {
  animal.makeSound();
}
```

Příklad polymorfismu

```
const dog = new Dog();
const cat = new Cat();

makeAnimalSound(dog); // "Woof!"
makeAnimalSound(cat); // "Meow!"
```

Funkce pracuje s typem `Animal`, ale volá správnou metodu podle skutečného typu!

Příklad polymorfismu

```
abstract class PaymentMethod {
  abstract processPayment(amount: number): void;
}
```

Příklad polymorfismu

```
class CreditCard extends PaymentMethod {
  processPayment(amount: number): void {
    console.log(`Paid ${amount} by credit card`);
  }
}

class PayPal extends PaymentMethod {
  processPayment(amount: number): void {
    console.log(`Paid ${amount} via PayPal`);
  }
}
```

Příklad polymorfismu

```
function checkout(
  payment: PaymentMethod,
  amount: number
): void {
  payment.processPayment(amount);
}

checkout(new CreditCard(), 100); // "Paid 100 by credit card"
checkout(new PayPal(), 50);      // "Paid 50 via PayPal"
```

Přetěžování metod

Přetěžování metod umožňuje definovat více verzí stejné funkce s různými parametry

TypeScript podporuje přetěžování pomocí **signatury funkcí**

Přetěžování metod

```
class Calculator {
  // Overload signatures
  add(a: number, b: number): number;
  add(a: string, b: string): string;
  add(a: number[], b: number[]): number[];

  // Implementation
  add(a: any, b: any): any {
```

Přetěžování metod

```
    if (typeof a === "number" && typeof b === "number") {
      return a + b;
    }
    if (typeof a === "string" && typeof b === "string") {
      return a + b;
    }
    if (Array.isArray(a) && Array.isArray(b)) {
      return [...a, ...b];
    }
  }
}
```

Přetěžování metod

```
const calc = new Calculator();

console.log(calc.add(1, 2));                // 3
console.log(calc.add("Hello", " World")); // "Hello World"
console.log(calc.add([1, 2], [3, 4])); // [1, 2, 3, 4]
```

Přetěžování s volitelnými parametry

```
class DateFormatter {
  format(date: Date): string;
  format(date: Date, format: string): string;
  format(date: Date, format?: string): string {
    if (format) {
      // Custom formatting
      return date.toLocaleDateString();
    }
    return date.toString();
  }
}
```

Polymorfismus s rozhraními

```
interface Shape {
  area(): number;
  perimeter(): number;
}
```

Polymorfismus s rozhraními

```
class Circle implements Shape {
  constructor(
    private radius: number
  ) {}

  area(): number { return Math.PI * this.radius ** 2; }
  perimeter(): number {
    return 2 * Math.PI * this.radius;
  }
}
```

Polymorfismus s rozhraními

```
class Rectangle implements Shape {
  constructor(
    private width: number,
    private height: number
  ) {}
  area(): number { return this.width * this.height; }
  perimeter(): number {
    return 2 * (this.width + this.height);
  }
}
```

Polymorfismus s rozhraními

```
function printShapeInfo(shape: Shape): void {
  console.log(`Area: ${shape.area()}`);
  console.log(`Perimeter: ${shape.perimeter()}`);
}

const circle = new Circle(5);
const rectangle = new Rectangle(4, 6);

printShapeInfo(circle);
printShapeInfo(rectangle);
```

Abstrakce a rozhraní

Abstraktní třídy

Abstraktní třída je třída, kterou nelze přímo instancovat

Slouží jako **základní šablona** pro jiné třídy

Používá se klíčové slovo `abstract`

Abstraktní třídy

```
abstract class Animal {
  constructor(public name: string) {}

  // Abstract method - must be implemented by child
  abstract makeSound(): void;

  // Concrete method - shared implementation
  move(): void {
    console.log(`${this.name} is moving`);
  }
}
```

Abstraktní třídy

```
// Cannot instantiate abstract class
const animal = new Animal("Generic"); // Error!
```

Abstraktní třídy

```
// Must extend and implement abstract methods
class Dog extends Animal {
  makeSound(): void {
    console.log("Woof!");
  }
}

const dog = new Dog("Rex"); // OK
dog.makeSound();            // "Woof!"
dog.move();                 // "Rex is moving"
```

Abstraktní třídy

Kdy používat abstraktní třídy?

- Chcete **sdílet implementaci** mezi třídami
- Chcete **vynutit určité metody** v potomcích
- Chcete definovat **společné chování**
- Máte **hierarchii** souvisejících tříd

Rozhraní

Rozhraní (interface) definuje **kontrakt** - strukturu objektu bez implementace

```
interface Person {
  name: string;
  age: number;
  greet(): string;
}
```

Rozhraní říká „co“, ne „jak“

Rozhraní

Použití rozhraní:

```
const john: Person = {
  name: "John",
  age: 30,
  greet() {
    return `Hello, I'm ${this.name}`;
  }
};

console.log(john.greet()); // "Hello, I'm John"
```

Rozhraní s volitelnými vlastnostmi

```
interface User {
  id: number;
  name: string;
  email?: string; // optional
  phone?: string; // optional
}

const user1: User = { id: 1, name: "John" }; // OK
const user2: User = {
  id: 2, name: "Jane", email: "jane@example.com"
}; // OK
```

Rozhraní s readonly vlastnostmi

```
interface Point {
  readonly x: number;
  readonly y: number;
}

const point: Point = { x: 10, y: 20 };
point.x = 5; // Error: Cannot assign to 'x' because it is a
read-only property
```

Rozhraní s readonly vlastnostmi

Rozšiřování rozhraní:

```
interface Person {
  name: string;
  age: number;
}

interface Employee extends Person {
  employeeId: number;
  department: string;
}
```

Rozhraní s readonly vlastnostmi

```
const emp: Employee = {
  name: "John",
  age: 30,
  employeeId: 1234,
  department: "IT"
};
```

Index Signatures - dynamické klíče

Index signatures umožňují definovat typ pro dynamické klíče objektu

Užitečné pro objekty, které fungují jako slovníky nebo mapy

Index Signatures - dynamické klíče

```
interface StringMap {
  [key: string]: string;
}

const translations: StringMap = {
  hello: "Ahoj",
  goodbye: "Sbohem",
  yes: "Ano"
};
```

Index Signatures - dynamické klíče

```
console.log(translations["hello"]); // "Ahoj"
translations["no"] = "Ne"; // OK
```

Pozor! JavaScript interně převádí čísla na řetězce, takže je možné použít i čísla jako klíče. TypeScript toto chování respektuje

```
translations[1] = "One"; // OK
console.log(translations[1]); // "One"
```

Index Signatures - dynamické klíče

Index signature s číselnými klíči:

```
interface NumberDictionary {
  [index: number]: string;
}

const names: NumberDictionary = {
  0: "Alice",
  1: "Bob",
  2: "Charlie"
};
```

Index Signatures - dynamické klíče

Kombinace fixních a dynamických vlastností:

```
interface Config {
  version: string;                    // fixed property
  [key: string]: string | number; // dynamic properties
}

const config: Config = {
  version: "1.0.0",
  port: 3000,
  host: "localhost"
};
```

Index Signatures - dynamické klíče

```
interface Config {
  version: string;
  [key: string]: string | number | Config;
}

const config: Config = {
  version: "1.0.0",
  port: 3000,
  host: "localhost",
  db: { version: "0.2.0", host: "localhost", port: 3306 }
};
```

Index Signatures - dynamické klíče

```
interface Cache<T> {
  [key: string]: T;
}

const userCache: Cache<{ name: string; age: number }> = {
  "user1": { name: "John", age: 30 },
  "user2": { name: "Jane", age: 25 }
};
```

Tento příklad využíval takzvaný **generický typ** (typový parametr), o kterém se ještě budeme bavit

Rozdíl mezi třídou a rozhraním

Kdy použít **rozhraní** a kdy **abstraktní třídu**?

Rozhraní:

- Pouze **definice** bez implementace
- Třída může implementovat **více rozhraní**
- **Čistý kontrakt** - co objekt umí
- Interface se používá pro **duck typing**
  - „Pokud to chodí jako kachna a kváká jako kachna, pak to musí být kachna“
- Hierarchie „má“ vztahů (has-a) / „umí“ (can-do)

Rozdíl mezi třídou a rozhraním

Abstraktní třída:

- Může obsahovat **implementaci**
- Třída může dědit pouze z **jedné** třídy
- **Sdílená funkcionalita** mezi potomky
- Hierarchie „je“ vztahů (is-a)

```
// Can implement multiple interfaces
class User implements Person, Serializable { }
// Can extend only one class
class Dog extends Animal { }
```

Rozdíl mezi třídou a rozhraním

```
// Interface - contract for functionality
interface Flyable {
  fly(): void;
}
```

Rozdíl mezi třídou a rozhraním

```
// Abstract class - shared behavior
abstract class Bird {
  constructor(public name: string) {}

  abstract makeSound(): void;

  eat(): void {
    console.log(`${this.name} is eating`);
  }
}
```

Rozdíl mezi třídou a rozhraním

```
// Class implementing interface and extending abstract class
class Sparrow extends Bird implements Flyable {
  makeSound(): void {
    console.log("Chirp!");
  }

  fly(): void {
    console.log(`${this.name} is flying`);
  }
}
```

Rozdíl mezi třídou a rozhraním

```
const sparrow = new Sparrow("Tweety");
sparrow.makeSound(); // "Chirp!"
sparrow.fly();       // "Tweety is flying"
sparrow.eat();       // "Tweety is eating"
```

Implementace rozhraní

Klíčové slovo `implements` se používá pro implementaci rozhraní

```
interface Logger {
  log(message: string): void;
  error(message: string): void;
}
```

Implementace rozhraní

```
class ConsoleLogger implements Logger {
  log(message: string): void {
    console.log(`[LOG] ${message}`);
  }

  error(message: string): void {
    console.error(`[ERROR] ${message}`);
  }
}
```

Implementace více rozhraní

```
interface Printable {
  print(): void;
}

interface Saveable {
  save(): void;
}
```

Implementace více rozhraní

```
class Document implements Printable, Saveable {
  print(): void {
    console.log("Printing document");
  }

  save(): void {
    console.log("Saving document");
  }
}
```

Rozhraní pro funkce

```
interface SearchFunc {
  (source: string, substring: string): boolean;
}

const mySearch: SearchFunc = function(src, sub) {
  return src.includes(sub);
};

console.log(mySearch("Hello World", "World")); // true
```

SOLID principy

Úvod do SOLID

SOLID je akronym pěti základních principů objektově orientovaného designu

Vytvořil je Robert C. Martin (Uncle Bob)

Cílem je vytvářet **udržovatelný**, **škálovatelný** a **flexibilní** kód

Úvod do SOLID

SOLID principy:

- **S** - Single Responsibility Principle
- **O** - Open/Closed Principle
- **L** - Liskov Substitution Principle
- **I** - Interface Segregation Principle
- **D** - Dependency Inversion Principle

Single Responsibility Principle (SRP)

Každá třída by měla mít pouze jednu odpovědnost (jeden důvod ke změně)

Třída by měla dělat pouze jednu věc a dělat ji dobře

Single Responsibility Principle (SRP)

❌ **Špatně** - třída má více odpovědností:

```
class User {
  constructor(
    public name: string,
    public email: string
  ) {}

  save() { }
  sendEmail() { }
}
```

Single Responsibility Principle (SRP)

✅ **Dobře** - každá třída má jednu odpovědnost:

```
class User {
  constructor(public name: string, public email: string) {}
}

const user = new User("John", "john@example.com");
```

Single Responsibility Principle (SRP)

```
class UserRepository {
  save(user: User) {
    console.log("Saving to database...");
  }
}

const repo = new UserRepository();
repo.save(user);
```

Single Responsibility Principle (SRP)

```
class EmailService {
  send(to: string, message: string) {
    console.log("Sending email...");
  }
}

const emailService = new EmailService();
emailService.send(user.email, "Welcome!");
```

Open/Closed Principle (OCP)

Třídy by měly být otevřené pro rozšíření, ale **uzavřené pro modifikaci**

Měli bychom být schopni přidat novou funkcionalitu bez změny existujícího kódu

Open/Closed Principle (OCP)

❌ **Špatně** - při přidání nového typu musíme modifikovat funkci:

```
class Rectangle {
  constructor(public width: number, public height: number) {}
}
class Circle {
  constructor(public radius: number) {}
}
```

Open/Closed Principle (OCP)

```
function calculateArea(shape: any): number {
  if (shape instanceof Rectangle) {
    return shape.width * shape.height;
  } else if (shape instanceof Circle) {
    return Math.PI * shape.radius ** 2;
  }
  return 0;
}
```

Co když přidáme trojúhelník? Musíme modifikovat `calculateArea`!

Open/Closed Principle (OCP)

✅ **Dobře** - použijeme rozhraní:

```
interface Shape {
  area(): number;
}
```

Open/Closed Principle (OCP)

```
class Rectangle implements Shape {
  constructor(
    private width: number,
    private height: number
  ) {}

  area(): number {
    return this.width * this.height;
  }
}
```

Open/Closed Principle (OCP)

```
class Circle implements Shape {
  constructor(
    private radius: number
  ) {}

  area(): number {
    return Math.PI * this.radius ** 2;
  }
}
```

Open/Closed Principle (OCP)

```
class Triangle implements Shape {
  constructor(
    private base: number,
    private height: number
  ) {}

  area(): number {
    return (this.base * this.height) / 2;
  }
}
```

Open/Closed Principle (OCP)

```
function calculateArea(shape: Shape): number {
  return shape.area();
}

// Can add new shapes without changing the calculateArea
const rect = new Rectangle(10, 5);
const circle = new Circle(5);
const triangle = new Triangle(10, 8);
```

Open/Closed Principle (OCP)

```
console.log(calculateArea(rect));      // 50
console.log(calculateArea(circle));    // 78.54
console.log(calculateArea(triangle)); // 40
```

Liskov Substitution Principle (LSP)

Objekty rodičovské třídy by měly být nahraditelné objekty potomků bez změny funkčnosti

Potomci musí dodržovat „smlouvu“ (kontrakt) rodičovské třídy

Liskov Substitution Principle (LSP)

❌ **Špatně** - porušení LSP:

```
class Bird {
  fly(): void { console.log("Flying..."); }
}

class Penguin extends Bird {
  fly(): void { throw new Error("Penguins can't fly!"); }
}
```

`Penguin` nelze použít místo `Bird` - porušuje očekávané chování!

Liskov Substitution Principle (LSP)

✅ **Dobře** - správná hierarchie:

```
abstract class Bird {
  abstract move(): void;
}
class FlyingBird extends Bird {
  move(): void { console.log("Flying..."); }
}
class Penguin extends Bird {
  move(): void { console.log("Swimming..."); }
}
```

Liskov Substitution Principle (LSP)

```
function makeBirdMove(bird: Bird): void {
  bird.move();
}

const sparrow = new FlyingBird();
const penguin = new Penguin();

makeBirdMove(sparrow); // "Flying..."
makeBirdMove(penguin); // "Swimming..."
```

Oba potomci fungují správně místo rodiče!

Interface Segregation Principle (ISP)

Klienti by neměli záviset na rozhraních, která nepoužívají

Lepší je mít více specifických rozhraní než jedno velké

Interface Segregation Principle (ISP)

❌ **Špatně** - velké rozhraní nutí implementovat nepotřebné metody:

```
interface Worker {
  work(): void;
  eat(): void;
  sleep(): void;
}
```

Interface Segregation Principle (ISP)

```
class Human implements Worker {
  work() { console.log("Working..."); }
  eat() { console.log("Eating..."); }
  sleep() { console.log("Sleeping..."); }
}
```

Interface Segregation Principle (ISP)

```
class Robot implements Worker {
  work() { console.log("Working..."); }
  eat() { /* Robot doesn't eat! */ }
  sleep() { /* Robot doesn't sleep! */ }
}
```

Robot musí implementovat metody, které nedávají smysl!

Interface Segregation Principle (ISP)

✅ **Dobře** - rozdělíme na menší rozhraní:

```
interface CanWork {
  work(): void;
}
interface CanEat {
  eat(): void;
}
interface CanSleep {
  sleep(): void;
}
```

Interface Segregation Principle (ISP)

```
class Human implements CanWork, CanEat, CanSleep {
  work() { console.log("Working..."); }
  eat() { console.log("Eating..."); }
  sleep() { console.log("Sleeping..."); }
}

class Robot implements CanWork {
  work() { console.log("Working..."); }
}
```

Každá třída implementuje jen to, co potřebuje!

Dependency Inversion Principle (DIP)

Závislosti by měly směřovat k abstrakcím, ne ke konkrétním implementacím

- High-level moduly by neměly záviset na low-level modulech
- Oba by měly záviset na abstrakcích (rozhraních)

Dependency Inversion Principle (DIP)

❌ **Špatně** - závislost na konkrétní implementaci:

```
class MySQLDatabase {
  save(data: string): void {
    console.log("Saving to MySQL:", data);
  }
}
```

Dependency Inversion Principle (DIP)

```
class UserService {
  private db = new MySQLDatabase();

  saveUser(name: string): void {
    this.db.save(name);
  }
}
```

Co když chceme použít PostgreSQL? Musíme měnit `UserService`!

Dependency Inversion Principle (DIP)

✅ **Dobře** - závislost na abstrakci:

```
interface Database {
  save(data: string): void;
}
```

Dependency Inversion Principle (DIP)

```
class MySQLDatabase implements Database {
  save(data: string): void {
    console.log("Saving to MySQL:", data);
  }
}

class PostgreSQLDatabase implements Database {
  save(data: string): void {
    console.log("Saving to PostgreSQL:", data);
  }
}
```

Dependency Inversion Principle (DIP)

```
class UserService {
  constructor(
    private db: Database
  ) {}

  saveUser(name: string): void {
    this.db.save(name);
  }
}
```

Dependency Inversion Principle (DIP)

```
// We can simply change the implementation
const mysqlService = new UserService(
  new MySQLDatabase()
);

const postgresService = new UserService(
  new PostgreSQLDatabase()
);
```

Dependency Injection - závislosti předáváme zvenčí!

Shrnutí SOLID

SOLID principy pomáhají vytvářet:

- **Udržovatelný** kód - snadná údržba a změny
- **Testovatelný** kód - snazší psaní testů
- **Flexibilní** kód - snadné rozšiřování
- **Znovupoužitelný** kód - komponenty lze použít jinde
- **Škálovatelný** kód - růst projektu bez problémů

Shrnutí SOLID

Klíčové zásady:

- **SRP**: Jedna třída = jedna odpovědnost
- **OCP**: Rozšiřuj, nemodifikuj
- **LSP**: Potomci musí být nahraditelní
- **ISP**: Malá rozhraní jsou lepší
- **DIP**: Závislosti na abstrakcích, ne implementacích

Závěr

Shrnutí

Co jsme se naučili:

Polymorfismus

- Stejné rozhraní, různá implementace
- Přepisování (overriding) a přetěžování (overloading) metod

Shrnutí

Abstrakce a rozhraní

- Abstraktní třídy
- Rozhraní (interfaces)
- Index Signatures
- Rozdíly a kdy co použít

Shrnutí

Příště nás čeká poslední díl TypeScriptu: **generika**, **dekorátory** a **mixiny**